

Report, Reflection and Key Finding

This project developed an interrupt security system in Tinkercad using an Arduino UNO. The sensing stage consist of the simulated three sensor a door button, window button, and a motion button. The thinking evaluates these inputs and selects one of the three states of SAFE, WARNING and ALRM. The actuation stage controls the alarm LED. The heartbeat on the built in LED provides a separates task.

Three different mechanisms were used, motion sensor uses `attachInterrupt()`, the door and window sensors use pin change interrupts because D8 and D9 belong to the same PORTB interrupt group. In `PCINT0_vect`, the current PORTB value is compared with the previous value using XOR. Where this identifies D8, D9 or both pins changed. Timer 1 operates in a clear timer on comparing match mode. Having the 16 MHz clock and the 1024 prescaler, the timer receives 15,625 counts per second, so that the OCR1A is set to 15,624 to generate one second interrupt.

The challenge that I faced was a mismatch between the circuit and the input logic. The tinkercad circuit uses an external pull-down resistor, where it released button produced LOW and pressed buttons produced HIGH. The program assumed internal pull up inputs, where it caused all sensors to appear active at startup. This was solved by configuring the pins as a normal INPUT and treating HIGH as the active situation.

Another challenge was keeping the interrupt service routine safe responsive. The serial output, state decision, and LED control were therefore kept out of the ISRs. Each of it only captures a state, increments a timer count and sets a volatile flag. The main loop safely copies shared information while `delay()`, and reduces the risk of multiple events from one button.

One of the findings was that the interrupts work best short event notification rather than location complete app logic. Separating event capture, decision making and actuation made the program easier to test and explain. The serial event labels and counters also showed that timer events continued while external and pin change events were handled, showing reliable separation and event driven behavior.